

A Soft Constraint Is Not Its Own Limit: Activation-Norm Penalties, Weight-Norm Ratchets, and the Geometry of Sequential Optimization

author names withheld

Under Review for OPT 2026

Abstract

We study what a soft penalty on hidden-activation norms does to the optimization dynamics of a network trained on a long sequence of tasks. The penalty pulls pre-activations toward a fixed-radius sphere and has a single hyperparameter. Tracking a battery of geometric diagnostics across a sweep of penalty strengths, we find that it arrests a cumulative inflation of both activation and weight norms that otherwise proceeds monotonically across tasks, and that retention of earlier tasks improves by 13.5 points at unchanged current-task accuracy. We give a decomposition of activation-space gradient flow that accounts for three of the observed effects, and we show that the reduction in radial gradient energy which the decomposition predicts arises for an unexpected reason: the radial gradient is essentially unchanged and the tangential gradient grows. Running through the paper is a comparison with the penalty’s own exactly-enforced limit. Projection makes the layer scale-invariant in its weights, so the loss gradient becomes orthogonal to the weight vector and the weight norm can only increase: the constraint holds exactly and the inflation it was meant to prevent moves into the weights. The hard-projection arm retains 19.9%, below an unpenalized baseline, against 34.2% for the soft penalty.

1. Introduction

Penalty and projection methods are usually presented as two routes to the same constrained optimum, differing in convergence rate rather than in the solution reached [6]. For a network trained by stochastic gradient descent this need not hold: the two constructions place different implicit biases on the trajectory, and in a sequential setting the trajectory determines how much previously fitted structure survives.

This paper characterizes one such penalty. Let $h \in \mathbb{R}^d$ be the pre-activation of a hidden layer. We add to the task loss

$$L_{\text{pen}}(h) = \frac{1}{d} (\|h\|_2 - \sqrt{d})^2, \quad (1)$$

with a single coefficient λ . The target $\sqrt{d}$ keeps the mean squared activation per unit at $\mathcal{O}(1)$, matching standard initialization. The same penalty has been studied in the context of delayed generalization on algorithmic tasks [2]; here we ask a different question, in a setting where the optimizer must repeatedly refit without destroying what it already has.

Our instrument is a long task sequence, which gives a direct readout of how much of a previously fitted solution an optimization path destroys. We are not proposing a continual-learning method and do not compete with replay or parameter-importance schemes; the sequence is a probe.

What we find.

1. Unpenalized training inflates both activation radius and weight norm *monotonically across tasks*. The penalty arrests both (§??).
2. The normalized radial gradient energy $\tilde{\Phi}_{\text{rad}}$ falls sharply at the first hidden layer and tracks retention across the sweep. It falls because the *tangential* gradient grows by 43.5%, not because the radial gradient shrinks, which it does by 4.9% (§3.1).
3. Reducing the learning rate reproduces the penalty’s entire static geometric signature (radius, radial excess, weight norm, effective rank) and reproduces neither its accuracy nor its retention (§4).
4. Exactly enforcing the constraint removes the benefit. We give a short argument for why: projection makes the layer degree-zero homogeneous in W , the loss gradient becomes orthogonal to W , and the weight norm becomes non-decreasing (§3.2).

2. Setup

Task sequence. Permuted MNIST, $T = 150$ tasks, a fresh input permutation per task, 10 epochs per task, no replay and no task identifier at test time. Network: 3-hidden-layer MLP of width 1000, ReLU, single head. SGD, learning rate 0.1, batch 256, global gradient-norm clip 0.5. The penalty is applied to pre-activations of all hidden layers, per sample, averaged over the batch. Unless stated otherwise, quantities are reported at layer 0, averaged over the final 20 tasks, over 3 seeds.

Metrics. We report current-task accuracy and *retention*, the mean accuracy over previously seen tasks, excluding the current one. Geometric diagnostics are computed at each task boundary on a fixed probe of 1000 task-0 inputs, held constant for the run. Per-layer quantities are never averaged across layers. Definitions in Appendix A.

Radial gradient energy. With $P_r = \hat{h}\hat{h}^\top$, $\hat{h} = h/\|h\|_2$, and $g = \nabla_h L_{\text{task}}$ the *task*-loss gradient only,

$$\tilde{\Phi}_{\text{rad}} = d \cdot \frac{\|P_r g\|_2^2}{\|g\|_2^2}, \quad \mathbb{E}[\tilde{\Phi}_{\text{rad}}] = 1 \text{ for isotropic } g. \quad (2)$$

The penalty gradient is excluded by construction: it is purely radial, so including it would make (2) partly a measurement of the regularizer.

3. Analytical framework

We give three propositions. Each is a statement about the gradient field, each yields a prediction, and each prediction is checked immediately.

3.1. Radial–angular decomposition and the relaxation timescale

Proposition 1 *The penalty gradient $\nabla_h L_{\text{pen}} = \frac{2}{d}(\|h\|_2 - \sqrt{d})\hat{h}$ is purely radial. Writing $u = \|h\|_2 - \sqrt{d}$ and $g_r = \langle \hat{h}, \nabla_h L_{\text{task}} \rangle$, the activation-space gradient flow $\dot{h} = -\nabla_h L_{\text{total}}$ separates as*

$$\dot{u} = -g_r - \frac{2\lambda}{d}u, \quad \dot{h}_{\text{tan}} = -(I - P_r)\nabla_h L_{\text{task}}. \quad (3)$$

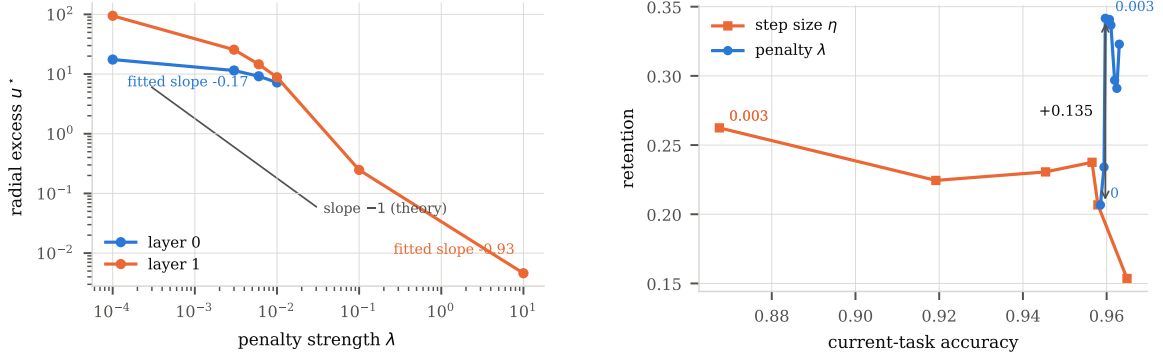


Figure 1: **Left:** steady-state radial excess against λ , per layer. Layer 1 follows the slope -1 predicted by (4); layer 0 is much flatter. **Right:** the two sweeps in the (accuracy, retention) plane. The step size traces a trade-off between the axes; the penalty sweep sits above it.

The radial coordinate is a first-order lag driven by g_r , with relaxation time and stationary point

$$\tau = \frac{d}{2\lambda}, \quad u^* = -\frac{d g_r}{2\lambda}. \quad (4)$$

As $\lambda \rightarrow \infty$ we have $\tau \rightarrow 0$, $u \rightarrow 0$, and the flow reduces to $\dot{h} = -(I - P_r)\nabla_h L_{\text{task}}$, which is Riemannian gradient flow on $\mathbb{S}^{d-1}(\sqrt{d})$ [1].

So λ does not only control how well the constraint is satisfied; it sets a timescale on which the radial coordinate forgets its history, and the exactly-enforced limit is the $\tau = 0$ member of that family rather than a separate construction.

Prediction 1a. $\log |u^*|$ against $\log \lambda$ has slope -1 , modulo any dependence of g_r on λ ; if $g_r \propto \lambda^a$ the slope is $a - 1$.

Result. Measured log-log slopes of steady-state radial excess are -0.17 ± 0.16 at layer 0 and -0.93 ± 0.31 at layer 1; layer 2 has too few positive-excess points to fit. Layer 1 is consistent with both the naive -1 and with the corrected $a - 1$; layer 0 is consistent with neither (Figure 1). (3) treats h as a free variable, whereas $h = Wx$ is constrained by the parameterization and by a fixed input distribution, and we take the layer-0 discrepancy as a failure of that idealization rather than of the decomposition.

Prediction 1b. The penalty suppresses $\tilde{\Phi}_{\text{rad}}$ below the unpenalized value, and the suppression tracks retention.

Result. At layer 0, $\tilde{\Phi}_{\text{rad}}$ falls from 0.392 at $\lambda = 0$ to 0.116 at $\lambda = 0.003$, and across the eight soft arms it is rank-correlated with retention at $\rho = -0.905$, $p = 0.002$. The relation is specific to the first hidden layer: at layers 1 and 2 the correlations are -0.14 and -0.17 , neither significant. Layer 1 is already close to tangential without any penalty ($\tilde{\Phi}_{\text{rad}} = 0.065$) and the penalty raises it to 0.106 . Table 1.

Table 1: Radial gradient energy per layer, and retention. $\tilde{\Phi}_{\text{rad}} = 1$ is the isotropic null. Layer 0 is the only layer whose $\tilde{\Phi}_{\text{rad}}$ tracks retention. Two interpolating rows omitted, full sweep in Appendix D. $n = 3$ except $\lambda = 0.1$ where $n = 2$.

Arm	Retention	Test acc.	$\tilde{\Phi}_{\text{rad}}^{(0)}$	$\tilde{\Phi}_{\text{rad}}^{(1)}$	$\tilde{\Phi}_{\text{rad}}^{(2)}$
$\lambda = 0$ (baseline)	0.2067	0.9585	0.392	0.065	0.684
$\lambda = 3 \cdot 10^{-3}$	0.3416	0.9597	0.116	0.061	0.423
$\lambda = 6 \cdot 10^{-3}$	0.3408	0.9607	0.106	0.073	0.394
$\lambda = 10^{-2}$	0.3367	0.9610	0.110	0.086	0.387
$\lambda = 10^{-1}$	0.3229	0.9630	0.145	0.100	0.322
$\lambda = 10$	0.2910	0.9624	0.186	0.106	0.318
Hard projection (exact)	0.1994	0.9580	0.000	0.031	0.446
Hard projection (STE)	0.2049	0.9588	1.035	0.197	0.574

The suppression is a denominator effect. Decomposing $\tilde{\Phi}_{\text{rad}} = d\|g_{\text{rad}}\|^2/\|g\|^2$ between $\lambda = 0$ and $\lambda = 0.003$ at layer 0: the radial gradient changes by -4.9% while the total task gradient changes by $+43.5\%$. The penalty does not suppress radial gradient energy. It amplifies tangential gradient energy, and $\tilde{\Phi}_{\text{rad}}$ falls as a consequence.

Proposition 2 (Desaturation) *Let $z = Vh$ be the logits and L_{task} cross-entropy, so that $\nabla_h L_{\text{task}} = V^\top(\text{softmax}(z) - y)$. For a correctly classified example, $\|\text{softmax}(z) - y\| \rightarrow 0$ as $\|z\| \rightarrow \infty$. Reducing $\|h\|$ reduces $\|z\|$, keeps the softmax away from saturation, and therefore increases $\|\nabla_h L_{\text{task}}\|$.*

This accounts for the $+43.5\%$ growth in the denominator and reframes what the penalty does: rather than damping motion in one direction, it keeps the task gradient alive in the remaining $d - 1$.

3.2. Scale invariance and the weight-norm ratchet

This is the point at which the penalty and its exactly-enforced limit come apart.

Proposition 3 *Let a layer compute $h = Wx$ followed by exact projection $\Pi(h) = \sqrt{d}h/\|h\|_2$. Then $\Pi(Wx)$ is invariant under $W \mapsto cW$ for every $c > 0$, so the layer’s contribution to the loss is degree-zero homogeneous in W . By Euler’s theorem,*

$$\langle \nabla_W L, W \rangle = 0, \quad (5)$$

and consequently a gradient step of size η satisfies

$$\|W - \eta \nabla_W L\|_F^2 = \|W\|_F^2 + \eta^2 \|\nabla_W L\|_F^2 \geq \|W\|_F^2. \quad (6)$$

The weight norm is non-decreasing under exact projection, whatever the loss. Under the soft penalty the corresponding inner product is

$$\langle \nabla_W L_{\text{pen}}, W \rangle = \frac{2\lambda}{d} (\|h\|_2 - \sqrt{d}) \|h\|_2, \quad (7)$$

which is strictly positive whenever $\|h\|_2 > \sqrt{d}$, so the descent step reduces $\|W\|_F$.

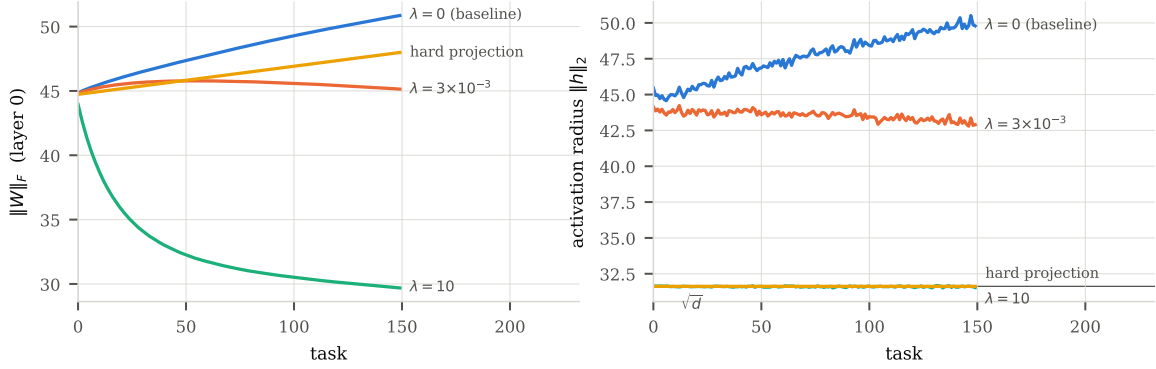


Figure 2: The ratchet, layer 0. **Left:** weight norm across the task sequence. **Right:** activation radius. Exact projection pins the radius to $\sqrt{d}$ and the weight norm climbs anyway; the soft penalty at $\lambda = 3 \cdot 10^{-3}$ arrests both. Plotted separately because the two are different measures on different scales.

The two constructions therefore differ in exactly the coordinate the constraint is about. Projection removes the radius from the forward pass, which makes weight growth free of charge; the soft penalty leaves the radius in the forward pass, so weight growth is still visible to the loss and is charged for. Equation (6) is the same mechanism by which scale-invariant layers under normalization schemes accumulate weight norm [3, 5, 7]; the observation here is that it converts an intended constraint into a ratchet.

Prediction 2. Under exact projection the activation radius is pinned and the weight norm nevertheless inflates across tasks. Under the soft penalty at small λ both are arrested.

Result. Figure 2; exact values in Appendix D. Over 150 tasks the unpenalized weight norm grows by $+6.03$. Exact projection pins the radius to $\sqrt{d}$ to six digits and still grows the weight norm by $+3.25$; the straight-through variant, whose backward pass is not the true gradient of a degree-zero map and therefore violates (5), grows it by $+7.63$, more than the baseline. The soft penalty at $\lambda = 0.003$ holds it to $+0.33$.

3.3. Anisotropic, task-rotating weight regularization

Proposition 4 Under $h = Wx$ and in the inflated regime $\|h\|_2 \gg \sqrt{d}$, where $(\|h\|_2 - \sqrt{d})^2 \approx \|h\|_2^2$,

$$\mathbb{E}_x[L_{\text{pen}}] \approx \frac{1}{d} \text{Tr}(W \Sigma_x W^\top), \quad \Sigma_x = \mathbb{E}_x[xx^\top]. \quad (8)$$

Unlike $\lambda \|W\|_F^2$ this weights directions by the variance of the inputs they read, and in a task sequence Σ_x changes with the task, so the axes of the regularizer rotate with it. The approximation holds early in each task and degrades as $\|h\|_2 \rightarrow \sqrt{d}$ (Appendix B).

Prediction 3. An isotropic weight penalty of matched strength cannot reproduce the benefit, because its axes do not rotate with the task.

Result. TODO S6: weight decay, EWC, SI, LayerNorm+WD.

Table 2: Same geometry, two ways. The step size reaches the penalty’s radius, radial excess, weight norm and effective rank, and arrives at a different solution. Intermediate rates in Appendix D. $n = 2$.

Arm	Retention	Test acc.	Radial exc.	Radius	$\ W\ _F$	Eff. rank
Penalty $\lambda = 3 \cdot 10^{-3}$	0.3416	0.9597	11.48	43.10	45.24	243.8
Step size $\eta = 3 \cdot 10^{-3}$	0.2624	0.8675	11.37	42.99	44.86	244.1
$\eta = 10^{-1}$ (default)	0.2068	0.9579	18.14	49.76	50.58	239.5
$\eta = 3 \cdot 10^{-1}$	0.1536	0.9649	35.97	67.59	68.34	235.8

4. The learning rate reaches the same geometry and not the same solution

A natural objection is that the penalty acts as an effective step-size reduction. The sweep over learning rates answers this without any matched construction. At $\eta = 0.003$ the unpenalized network reproduces the penalized network’s static geometry across every diagnostic we log, and reproduces neither its accuracy nor its retention (Table 2).

Across the learning-rate sweep, retention falls as η rises and accuracy rises with it ($\rho = -0.77$, $p = 0.07$): the step size trades one axis against the other. The penalty sweep lies above that trade-off by up to 0.169 in retention at matched accuracy (Figure 1, right).

5. Magnitude-matched controls

To separate magnitude from direction we ran three interventions that replay a recorded aspect of the penalized run onto an unpenalized one, per seed and per step: its realized per-group gradient magnitude, its total gradient magnitude, and its per-group weight-norm trajectory. Each arm is gated on a pre-registered matching tolerance (median $|\log \text{ratio}| < 0.05$) which is checked before any outcome is examined; an arm that fails is not reported. Details in Appendix C.

The ordering is monotone across the battery (Table 3, appendix): the control reproducing more of the signature reproduces more of the effect. Matching gradient magnitude recovers -1% and -7% of the retention gain and *raises* $\tilde{\Phi}_{\text{rad}}$ above baseline; matching the weight-norm trajectory reproduces the penalized weight norm to 0.5% , moves $\tilde{\Phi}_{\text{rad}}$ partway, and recovers 16% .

6. The battery

Across the eight soft arms, three diagnostics track retention at layer 0: $\tilde{\Phi}_{\text{rad}}$, cumulative representation drift from task 0, and the end-of-task update norm, all at $\rho = -0.905$, $p = 0.002$. Consecutive drift and its tangential part track more weakly ($\rho = -0.738$, $p = 0.037$). Radial drift, effective and stable rank, subspace overlap and gradient readiness do not track at all; the full table is Appendix D. We report the negatives because they constrain the account as much as the positives. **TODO: add curvature row (Hessian trace via Hutchinson) and σ_{max} .**

Two negatives matter. The dead-unit fraction is 0.000 in every arm, so loss of plasticity in the sense of Dohare et al. [4] is absent and forgetting is the only failure mode present. Subspace over-

lap does not track retention in either form, ruling out the most natural representation-preservation account.

TODO: per-neuron task selectivity figure, requires end-of-run checkpoints.

7. Optimizer interaction

TODO S7. The decomposition in Proposition 1 is stated for gradient flow. A diagonal preconditioner rescales coordinates independently and does not in general preserve the radial/tangential split in activation space, so the effect should weaken under adaptive methods and survive under momentum. Table with SGD / SGD+momentum / Adam, with and without penalty, learning rate chosen per optimizer and stated.

8. Limitations

Proposition 3 is the only part of the account that is derived rather than inferred. The rest is correlational: $\tilde{\Phi}_{\text{rad}}$ tracks retention across the sweep, but the sweep is one-dimensional in λ and eight aggregate points cannot separate $\tilde{\Phi}_{\text{rad}}$ from λ . The intervention that would separate them, an arm that lowers $\tilde{\Phi}_{\text{rad}}$ without constraining the radius, we have not run. Proposition 4 holds only in the inflated regime and describes early training within each task rather than the whole trajectory. Equation (3) treats h as free, and the measured equilibrium slope matches at layer 1 but not at layer 0, which we attribute to the weight parameterization. Finally, this is one task family, one architecture, 3 seeds, and one optimizer as the primary setting. We do not claim universality; the sequence is an instrument for observing optimization geometry, not a benchmark to compete on. TODO: second task family.

9. Conclusion

A soft activation-norm penalty and its exactly-enforced limit are not two routes to one solution. The penalty arrests a cumulative inflation of activation and weight norms across tasks; the projection pins the activation radius, makes the layer scale-invariant in its weights, and lets the same inflation proceed in weight space, where nothing charges for it. The useful settings of λ are the ones at which the constraint is barely enforced, and the quantity that tracks the benefit is not how well the constraint holds but how much tangential gradient survives.

References

- [1] P.-A. Absil, R. Mahony, and R. Sepulchre. Optimization algorithms on matrix manifolds. Princeton University Press, 2008.
- [2] Anonymous. Radial suppression accelerates algorithmic generalization: A geometric analysis of delayed generalization. In *ICML Workshop (non-archival)*, 2026.
- [3] Sanjeev Arora, Zhiyuan Li, and Kaifeng Lyu. Theoretical analysis of auto rate-tuning by batch normalization. In *ICLR*, 2019.

- [4] Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mahmood, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632, 2024.
- [5] Zhiyuan Li and Sanjeev Arora. Reconciling modern deep learning with traditional optimization analyses: The intrinsic learning rate. In *NeurIPS*, 2020.
- [6] Jorge Nocedal and Stephen J. Wright. Numerical optimization. Springer, 2006.
- [7] Twan van Laarhoven. L2 regularization versus batch and weight normalization. 2017.

Appendix A. Metric definitions

All quantities are computed at each task boundary and kept per layer. h denotes pre-activations of the layer in question, d its width.

Accuracy and retention. Let $a_t^{(k)}$ be accuracy on task k 's held-out set after finishing task t , evaluated on the first 2000 test examples of that task. We report current-task accuracy $a_t^{(t)}$ and retention $\frac{1}{t} \sum_{k < t} a_t^{(k)}$, which excludes the current task and is undefined at $t = 0$. We also log $\frac{1}{t+1} \sum_{k \leq t} a_t^{(k)}$, which includes it; the two differ by $\mathcal{O}(1/t)$ and the second is biased toward arms with higher current-task accuracy, which is why it is not the reported metric.

Radial quantities. Radius $\|h\|_2$ and radial excess $\|h\|_2 - \sqrt{d}$, both averaged over the probe batch; `radius_std` is the standard deviation of $\|h\|_2$ across probe inputs, which is identically zero under exact projection.

Radial gradient energy. $\tilde{\Phi}_{\text{rad}}$ as in (2), with g obtained by `autograd.grad` of the task loss with respect to pre-activations. The penalty is never included in the probe loss. A regression test asserts that passing the total gradient instead changes the value, and that passing the task gradient reproduces the $\lambda = 0$ value exactly.

Drift. On a probe of 1000 task-0 inputs, fixed for the run. For representations $h_{\text{cur}}, h_{\text{ref}}$ we log the absolute displacement $\|h_{\text{cur}} - h_{\text{ref}}\|_F$, its radial and tangential parts with respect to h_{ref} , and each divided by $\|h_{\text{ref}}\|_F$. Two references are used: the previous task boundary (*consecutive*) and the end of task 0 (*cumulative*). The relative forms are reported for comparability with prior work but are not used for cross- λ comparison, since the denominator is a quantity the penalty directly shrinks.

Subspace overlap. Top- k left singular subspaces of the two representation matrices with $k = 50$ fixed for every arm; overlap is the mean cosine of the principal angles. Bases are computed in float64 on CPU, which is deterministic and does not fail on the ill-conditioned matrices that arise at large λ .

Rank and units. Effective rank $\exp(-\sum_i p_i \log p_i)$ with $p_i = \sigma_i / \sum_j \sigma_j$; stable rank $\|A\|_F^2 / \|A\|_2^2$; dead fraction, the proportion of units with zero output on every probe input; dormant fraction, the proportion whose mean absolute activation is below 0.025 of the layer mean.

Readiness. Over $m = 8$ microbatches of the probe, $\|\bar{g}\|_2 \cdot \|\bar{g}\|_2^2 / \overline{\|g_i\|_2^2}$, computed *per layer*.

Appendix B. Derivations

B.1. Proposition 1

With $r = \|h\|_2$ and $\hat{h} = h/r$, $\nabla_h L_{\text{pen}} = \frac{2}{d}(r - \sqrt{d})\hat{h}$, which lies along $\hat{h}$. Writing $u = r - \sqrt{d}$ and projecting $\dot{h} = -\nabla_h L_{\text{total}}$ onto $\hat{h}$ gives (3). It is linear in u with coefficient $-2\lambda/d$, hence relaxation time $\tau = d/2\lambda$ and stationary point $u^* = -dg_r/2\lambda$. As $\lambda \rightarrow \infty$ the radial mode is infinitely stiff, $u \rightarrow 0$, and only the tangential term survives, which is gradient flow restricted to $\mathbb{S}^{d-1}(\sqrt{d})$.

The idealization is that h is treated as a free variable. In the network $h = Wx$, so its reachable set is constrained by the parameterization and by the input distribution, and the descent is on W

rather than on h . We report the measured slopes against this caveat rather than claiming the ODE governs the network.

B.2. Proposition 3

$\Pi(cWx) = \sqrt{d}cWx/\|cWx\|_2 = \sqrt{d}Wx/\|Wx\|_2 = \Pi(Wx)$ for $c > 0$, so the layer’s contribution to L satisfies $L(cW) = L(W)$. Differentiating in c at $c = 1$ gives $\langle \nabla_W L, W \rangle = 0$, which is Euler’s theorem for a degree-zero homogeneous function. Then

$$\|W - \eta \nabla_W L\|_F^2 = \|W\|_F^2 - 2\eta \langle \nabla_W L, W \rangle + \eta^2 \|\nabla_W L\|_F^2 = \|W\|_F^2 + \eta^2 \|\nabla_W L\|_F^2,$$

which is (6). The growth per step is second order in η and is never opposed, so $\|W\|_F$ increases monotonically. This is the mechanism by which scale-invariant layers accumulate weight norm under normalization [3, 7]; the point here is that it also applies when the invariance is introduced by a constraint intended to prevent growth.

For the soft penalty, $\nabla_W L_{\text{pen}} = \frac{2}{d}(r - \sqrt{d})\hat{h}x^\top$ and $\langle \hat{h}x^\top, W \rangle = \hat{h}^\top Wx = \hat{h}^\top h = r$, giving (7). Two caveats: the argument is for a single layer with the projection applied to its own output, and the straight-through variant does not satisfy (5) because its backward pass is not the Jacobian of a degree-zero map, which is consistent with its inflating faster than the exact version.

B.3. Proposition 2

For cross-entropy with logits $z = Vh$, $\nabla_h L = V^\top(\text{softmax}(z) - y)$. On a correctly classified example $\text{softmax}(z) \rightarrow y$ as the correct logit margin grows, so the residual and hence $\|\nabla_h L\|$ vanish. Scaling h down scales z down and moves the softmax away from that regime. The statement is qualitative: it predicts the sign of the change in $\|\nabla_h L\|$, not its size.

B.4. Proposition 4

If $\|h\|_2 \gg \sqrt{d}$ then $(\|h\|_2 - \sqrt{d})^2 = \|h\|_2^2(1 - \sqrt{d}/\|h\|_2)^2 = \|h\|_2^2(1 + \mathcal{O}(\sqrt{d}/\|h\|_2))$, and $\mathbb{E}_x \|Wx\|_2^2 = \text{Tr}(W\Sigma_x W^\top)$ gives (8). The relative error is $\mathcal{O}(\sqrt{d}/\|h\|_2)$, which at the measured $\lambda = 0$ radius of 49.8 against $\sqrt{d} = 31.6$ is roughly 63%: the approximation is loose even where it is best, and we use it only to establish the *form* of the regularizer, not its magnitude.

Appendix C. Control construction and acceptance tests

TODO: matching procedure, per-seed acceptance reports.

Appendix D. Full battery

TODO: add per-layer columns and the curvature diagnostics.

Appendix E. Experimental detail and reproducibility

Configuration. Permuted MNIST, 150 tasks, one fresh input permutation per task drawn from the run seed, 10 epochs per task, batch 256. MLP with 3 hidden layers of width 1000, ReLU, Kaiming uniform initialization, single 10-way head shared across tasks, no task identifier at test time and no replay. SGD without momentum, learning rate 0.1, global gradient-norm clip 0.5 except where stated. The penalty is applied to pre-activations of all hidden layers.

Table 3: Controls ordered by how much of the penalized signature they reproduce. Clipping is relaxed here (1.0 rather than 0.5) so that a global magnitude difference exists to be matched; under tight clipping both arms are pinned to the same total gradient norm and the comparison is vacuous. $n = 3$.

Arm	Retention	$\tilde{\Phi}_{\text{rad}}$	Radius	$\ W\ _F$	Cum. drift
Penalty $\lambda = 3 \cdot 10^{-3}$	0.3016	0.102	45.92	48.01	1175
Weight-norm matched	0.2210	0.190	48.54	48.25	1253
Baseline	0.2062	0.212	58.51	59.04	1365
Gradient matched, per group	0.2040	0.355	58.72	58.94	1354
Gradient matched, global	0.1971	0.346	57.02	57.61	1299

Table 4: The ratchet, exact values for Figure 2. Layer 0, first task to last.

Arm	$\ W\ _F : t_0 \rightarrow t_{149}$	radius: $t_0 \rightarrow t_{149}$	Retention
$\lambda = 0$ (baseline)	44.83 $\rightarrow$ 50.86 (+6.03)	45.47 $\rightarrow$ 49.79 (+4.32)	0.2067
$\lambda = 3 \cdot 10^{-3}$	44.81 $\rightarrow$ 45.14 (+0.33)	44.14 $\rightarrow$ 42.90 (−1.25)	0.3416
$\lambda = 10$	43.96 $\rightarrow$ 29.69 (−14.27)	31.64 $\rightarrow$ 31.54 (−0.11)	0.2910
Hard projection (exact)	44.74 $\rightarrow$ 47.99 (+3.25)	31.62 $\rightarrow$ 31.62 (+0.00)	0.1994
Hard projection (STE)	45.10 $\rightarrow$ 52.73 (+7.63)	31.62 $\rightarrow$ 31.62 (+0.00)	0.2049

Table 5: Rank correlation with retention across the eight soft arms, layer 0. Quantities that do not track are reported as such.

Diagnostic	ρ	p	Diagnostic	ρ	p
$\tilde{\Phi}_{\text{rad}}$	−0.905	0.002	radial drift	+0.19	n.s.
cumulative drift from t_0	−0.905	0.002	effective rank	+0.57	n.s.
update norm	−0.905	0.002	stable rank	+0.57	n.s.
consecutive drift	−0.738	0.037	subspace overlap	+0.29	n.s.
tangential drift	−0.738	0.037	gradient readiness	+0.17	n.s.

Seeds and reporting. 3 seeds per arm. Every table states n . An arm that has not reached its planned seed count is not reported; comparisons are paired on seed and give the test statistic, p , n and the sign split. Rank correlations across arms are Spearman.

Gradient clipping and its interaction with the controls. Under the default clip of 0.5 the clip binds on 89% of steps and pins both the penalized and unpenalized arms to the same total gradient norm, so the global magnitude confound is zero by construction and a global-magnitude control has nothing to remove. The control experiments therefore use a relaxed clip of 1.0, which binds on 46% of steps and leaves a global magnitude difference of 20.1%. Removing clipping entirely is not an option: at $\eta = 0.1$ the run diverges, with activation radius reaching 253 by task 10.

Ordering of gradient operations. Any rescaling applied to gradients is applied *after* clipping. Applied before, the clip renormalizes it away and the intervention silently does nothing. The ordering is asserted at run time and covered by a regression test.

Numerics. All spectral quantities are computed on CPU in float64. The GPU decomposition is not used: it raises on ill-conditioned activation matrices and its internal fallback is orders of magnitude slower. Any failure returns NaN and increments a counter that is written into the run’s configuration record, so a run with degraded diagnostics is identifiable.

Records. Each run writes its complete configuration before training starts, including every constant that is otherwise implicit (batch size, clip norm, probe size, method coefficients), a schema version, the code revision, and the hash of any external input file it consumes. `Code and run records will be released.`